ML-KEM (Kyber) in C

Implementation and benchmark report

NIST FIPS 203 — ML-KEM-512, ML-KEM-768, ML-KEM-1024 | 9 November 2026

1. Summary

This report describes a from-scratch implementation of the ML-KEM key-encapsulation mechanism (the standardised form of CRYSTALS-Kyber) in portable C99, and a systematic benchmark of its three parameter sets. The implementation follows FIPS 203, including the Fujisaki-Okamoto (FO) transform with implicit rejection, and is checked against an independent implementation.

- **Correctness.** 712 automated checks pass, including 60 known-answer vectors (20 per parameter set) generated by an independent pure-Python implementation: public key, secret key, ciphertext and shared secret match byte for byte. The tests also run clean under AddressSanitizer and UndefinedBehaviorSanitizer.
- **Speed.** With GCC 13.3 at -O3, portable C, one core: KeyGen / Encaps / Decaps take 33.1 / 36.1 / 47.0 us for ML-KEM-512, 51.8 / 56.3 / 70.8 us for ML-KEM-768 and 81.9 / 84.8 / 104.5 us for ML-KEM-1024 (medians).
- **Scaling.** Going from category 1 to category 5 costs about 2.5x in KeyGen time; the cost grows with the module rank k, but less than the k^2 growth of the matrix.
- **Where the time goes.** Expanding the public matrix with SHAKE128 takes 34% (ML-KEM-512) to 56% (ML-KEM-1024) of KeyGen; Keccak-f[1600] is the dominant primitive.
- **Caveat.** Numbers come from a shared cloud virtual machine, so absolute values are indicative; the relative comparisons (scaling, breakdown, compiler flags) are the robust findings. Section 8 lists the limitations.

2. The scheme and its parameters

ML-KEM turns an IND-CPA secure lattice-based public-key encryption scheme (K-PKE, based on Module-LWE) into an IND-CCA2 secure KEM using the Fujisaki-Okamoto transform. The hash functions are H = SHA3-256, G = SHA3-512, J = SHAKE256, and the matrix is expanded with SHAKE128.

Step	Computation
KeyGen (Alice)	(ek, s) = PKE.KeyGen(); z random 256 bits; dk = (s, ek, H(ek), z); return ek, dk
Encaps (Bob)	m random 256 bits; (K, R) = G(m, H(ek)); c = PKE.Enc(ek, m; R); return c, K
Decaps (Alice)	m' = PKE.Dec(s, c); (K', R') = G(m', H(ek)); Kbar = J(z, c); c' = PKE.Enc(ek, m'; R'); if c != c' return Kbar (implicit rejection), else return K'

Table 1. The FO transform as implemented (ML-KEM.KeyGen, Encaps, Decaps).

Scheme	NIST cat.	k	eta1, eta2	du, dv	delta	ek (B)	dk (B)	ct (B)
ML-KEM-512	1	2	3, 2	10, 4	2^{-139}	800	1632	768
ML-KEM-768	3	3	2, 2	10, 4	2^{-164}	1184	2400	1088
ML-KEM-1024	5	4	2, 2	11, 5	2^{-174}	1568	3168	1568

Table 2. Parameter sets. Common to all: n = 256, q = 3329, seed and message 32 bytes, shared secret 32 bytes. delta is the decryption failure probability. Categories 1, 3, 5 target security comparable to AES-128, AES-192, AES-256.

3. Implementation and verification

- **Language and layout.** Portable C99, no dependencies. Modules: Keccak/SHA-3/SHAKE (fips202), NTT and polynomial arithmetic, K-PKE, the FO-transform KEM, and an OS random-number wrapper. One runtime parameter object selects the security level.
- **Arithmetic.** Coefficients are 16-bit values in [0, q). Modular reduction uses a fixed-sequence Barrett reduction; the 7-layer NTT and the pairwise base-case multiplication follow FIPS 203, Algorithms 9-11.

Implementation and report prepared with assistance from Claude (Anthropic, model Claude Sonnet 5.5).

- **Side-channel hygiene.** No secret-dependent branches or table indices; the re-encryption comparison and the choice between K' and Kbar are constant-time; secrets are wiped after use. This is by design and not formally verified.
- **Validation.** SHA-3/SHAKE known answers; NTT round trip; 60 KATs against an independent implementation; 500 randomised round trips per parameter set; tampered-ciphertext tests confirming the output equals SHAKE256(z || c); and input checks (non-canonical encapsulation key, inconsistent decapsulation key).

4. Benchmark methodology

Item	Value
CPU	Intel Xeon @ 2.10 GHz, 2 vCPUs (1 thread per core), cloud virtual machine
Caches (as reported)	L1d 96 KiB, L2 4 MiB, L3 260 MiB
OS / compiler	Linux 6.18, x86-64; GCC 13.3.0, flags -std=c99 -O3 -Wall -Wextra -Wpedantic
Process control	pinned to one core (sched_setaffinity); CPU frequency cannot be controlled from inside the VM
Timer	clock_gettime(CLOCK_MONOTONIC), overhead about 29 ns per call pair; x86 time-stamp counter (TSC) read alongside
Sampling	10,000 samples per operation per run, 500 warm-up iterations discarded, 3 independent runs
Batching	KEM operations timed one call at a time; micro-operations (NTT, Keccak) timed in batches of 200 and divided
Randomness	excluded: the deterministic entry points are timed, with the input changed on every call

Table 3. Benchmark environment and settings.

Each sample is one timed call. For every operation the harness reports the minimum, median, mean, standard deviation and 95th percentile. The **median** is the headline figure because it is robust against scheduler and noisy-neighbour interference; the minimum approximates the undisturbed best case. The numbers in this report are the **median of the three per-run medians**, and the error bars in the figures span the lowest and highest of those three medians. Cycle counts are TSC ticks, which run at the nominal 2.1 GHz and not at the actual core frequency, so they are indicative only.

5. Results

Scheme	Operation	Median (us)	Min (us)	p95 (us)	TSC ticks (median)	Ops/s (median)
ML-KEM-512	KeyGen	33.12	31.27	55.5	69,643	30,198
	Encaps	36.09	34.12	63.5	75,891	27,709
	Decaps	46.99	44.19	79.0	98,795	21,280
ML-KEM-768	KeyGen	51.82	49.16	80.6	108,908	19,299
	Encaps	56.33	54.00	87.9	118,402	17,753
	Decaps	70.80	67.75	105.8	148,794	14,124
ML-KEM-1024	KeyGen	81.86	76.71	139.8	172,006	12,216
	Encaps	84.81	82.36	144.9	178,210	11,791
	Decaps	104.53	95.53	182.2	219,612	9,567

Table 4. Main results at -O3 (median of three runs; min = lowest observed over all runs; p95 averaged over runs).

Implementation and report prepared with assistance from Claude (Anthropic, model Claude Sonnet 5.5).

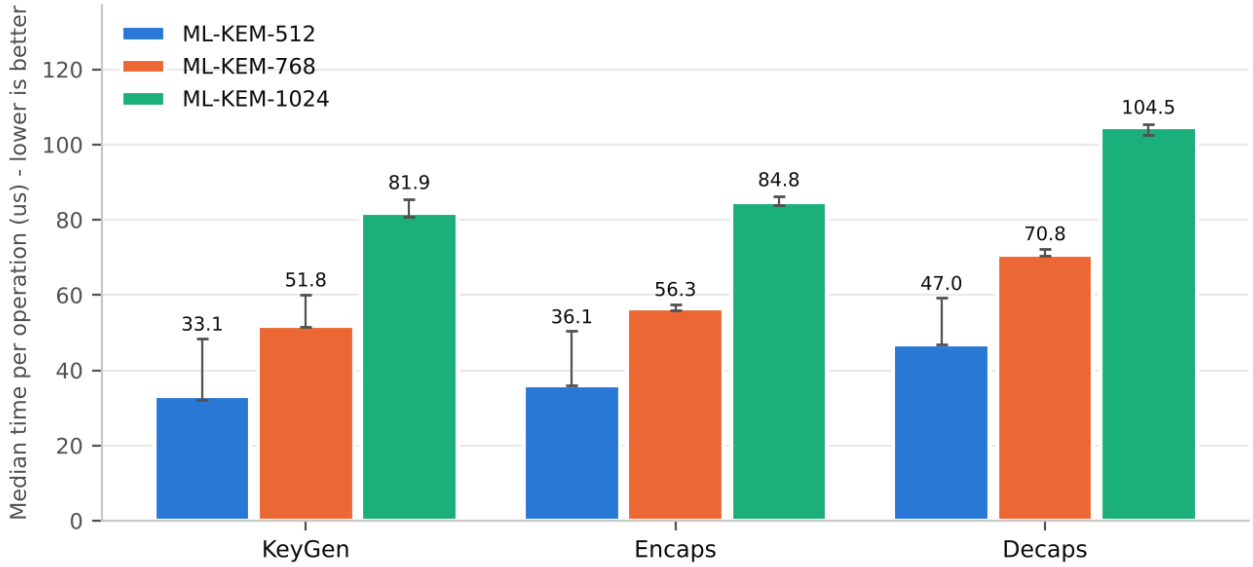


Figure 1. Median time per operation. Error bars show the range of the three per-run medians. The wide bars for ML-KEM-512 reflect one disturbed run on the shared host (the first run was 45-50% slower than the other two); the median is unaffected.

5.1 Scaling with the security level

Moving from ML-KEM-512 to ML-KEM-1024 doubles k but quadruples the number of matrix entries (k^2: 4, 9, 16). Total time grows by 2.47x (KeyGen), 2.35x (Encaps) and 2.22x (Decaps). That is well below 4x because the part of the work that grows with k^2 (matrix expansion and matrix-vector products) is only a share of the total, and Decaps also pays a fixed cost for its extra hashing.

Operation	512 (us)	768 (us)	1024 (us)	768 / 512	1024 / 512
KeyGen	33.1	51.8	81.9	1.56x	2.47x
Encaps	36.1	56.3	84.8	1.56x	2.35x
Decaps	47.0	70.8	104.5	1.51x	2.22x

Table 5. Relative cost of the three security levels (medians at -O3).

6. Where the time goes

Building block	Median	Notes
Keccak-f[1600] permutation	0.68 us (1,439 ticks)	the core of SHA-3 / SHAKE
NTT (forward)	1.65 us	one 256-coefficient polynomial
Inverse NTT	1.75 us	includes the final scaling by 128^-1
Expand matrix, k = 2	11.27 us	4 polynomials, 2.82 us each
Expand matrix, k = 3	25.58 us	9 polynomials, 2.84 us each
Expand matrix, k = 4	45.86 us	16 polynomials, 2.87 us each

Table 6. Building-block timings (-O3).

Sampling one matrix polynomial needs at least three Keccak-f permutations, because one SHAKE128 block (168 bytes) yields roughly 91 accepted 12-bit coefficients after rejection of values at or above q and 256 are required. At 0.68 us per permutation that is at least 2.05 us of the measured 2.84 us per polynomial, i.e. roughly three quarters. Hashing therefore dominates matrix expansion, and matrix expansion dominates the schemes (Figure 2).

Implementation and report prepared with assistance from Claude (Anthropic, model Claude Sonnet 5.5).

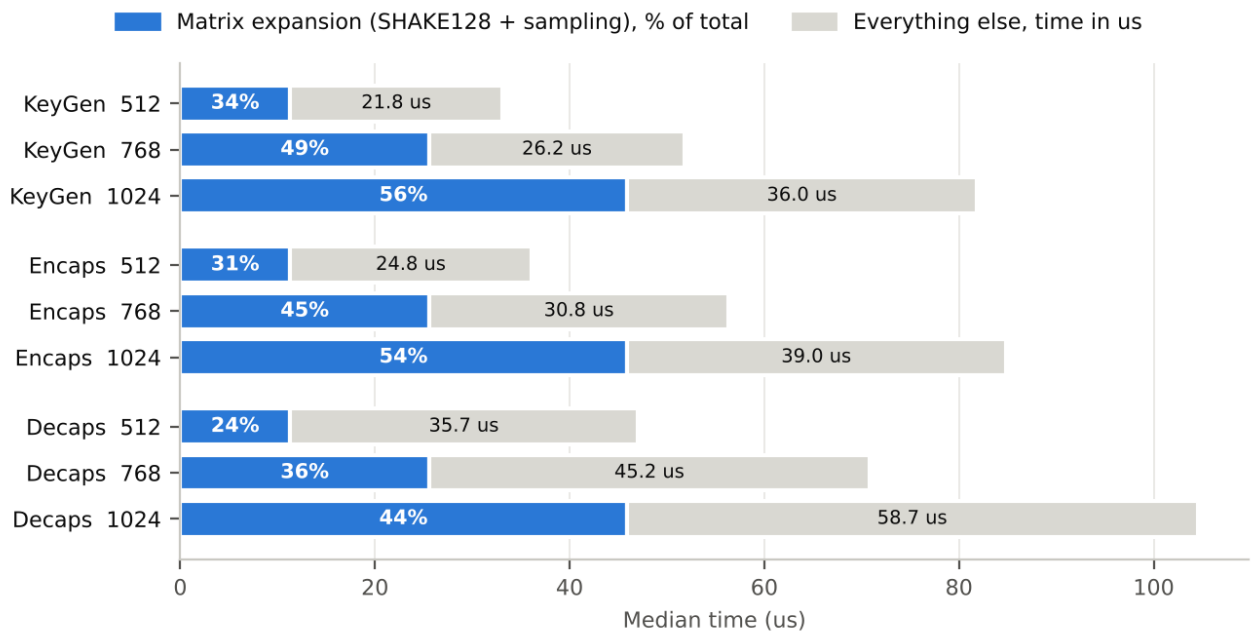


Figure 2. Share of each operation spent expanding the public matrix A (in KeyGen directly; in Encaps once; in Decaps once during re-encryption).

The number-theoretic transforms are a smaller share: KeyGen needs 2k forward NTTs, Encaps needs k forward and k+1 inverse, and Decaps (decryption plus re-encryption) needs 2k forward and k+2 inverse. Using the measured per-transform costs, this is about 19% of KeyGen, 21% of Encaps and 26% of Decaps for ML-KEM-768. The remainder is SHAKE256 noise sampling, SHA-3 hashing of the keys and ciphertexts, base-case multiplications, and (de)compression and byte encoding.

7. Further checks

7.1 Valid versus invalid ciphertext (implicit rejection)

Scheme	Valid median (us)	Invalid median (us)	Ratio	Valid min (us)	Invalid min (us)	Ratio
ML-KEM-512	46.99	46.43	0.988	44.19	44.21	1.000
ML-KEM-768	70.80	72.25	1.021	67.75	67.75	1.000
ML-KEM-1024	104.53	107.01	1.024	95.53	97.90	1.025

Table 7. Decapsulation of a valid ciphertext versus one with a flipped bit (which triggers implicit rejection).

Both paths do the same work (decrypt, re-encrypt, compare, select), and the measured ratios lie between 0.988 and 1.025. This is consistent with the absence of a secret-dependent early exit, but it is a timing sanity check on a noisy machine and not a statistical leakage test; tools such as dudect or ctgrind would be needed for a real constant-time claim.

Implementation and report prepared with assistance from Claude (Anthropic, model Claude Sonnet 5.5).

7.2 Compiler optimisation level

Scheme	Operation	-O2 (us)	-O3 (us)	-O3 -march=native (us)	-O2 / -O3	native / -O3
ML-KEM-512	KeyGen	57.0	33.1	30.9	1.72x	0.93x
	Encaps	62.7	36.1	32.6	1.74x	0.90x
	Decaps	75.3	47.0	41.6	1.60x	0.89x
ML-KEM-768	KeyGen	91.4	51.8	52.1	1.76x	1.01x
	Encaps	98.4	56.3	65.6	1.75x	1.16x
	Decaps	118.6	70.8	68.2	1.68x	0.96x
ML-KEM-1024	KeyGen	144.0	81.9	88.5	1.76x	1.08x
	Encaps	150.7	84.8	85.2	1.78x	1.00x
	Decaps	181.4	104.5	114.3	1.74x	1.09x

Table 8. Median of three runs for each build configuration.

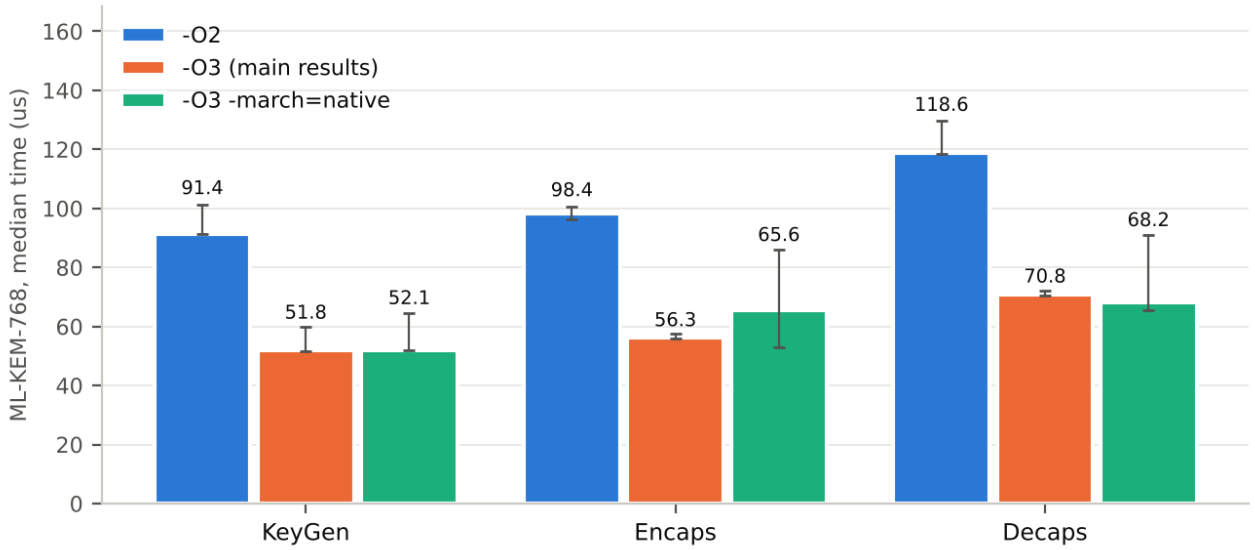


Figure 3. Effect of compiler flags on ML-KEM-768. Error bars show the range of three per-run medians.

-O3 is consistently faster than -O2, by 1.60x to 1.78x across all nine cells. Adding -march=native changed medians by between -11% and +16% relative to -O3 with no consistent direction, and its per-run spread is large (Figure 3). On this shared machine the effect of -march=native is therefore **inconclusive**, and the -O3 build is used for the main results.

8. Limitations

- **Shared cloud VM.** Other tenants on the host cause run-to-run noise (one run of ML-KEM-512 was about 45% slower); CPU frequency cannot be fixed. Medians over three runs reduce, but do not remove, this. Absolute times will differ on other hardware.
- **Portable C, no SIMD.** This is a clear, portable implementation. Optimised AVX2 or assembly implementations are known to be several times faster, so these figures should not be compared directly with them.
- **Cycle counts** are nominal TSC ticks, not core cycles.
- **Security scope.** Checked against one independent implementation and a self-written test suite; not audited, not NIST-ACVP-validated, and constant-time behaviour is not formally verified. It is intended for study, not for production use.

Implementation and report prepared with assistance from Claude (Anthropic, model Claude Sonnet 5.5).

9. Conclusion

The C implementation of ML-KEM passes all functional checks, including byte-exact agreement with an independent implementation on 60 vectors, and runs a full KeyGen, Encaps and Decaps in about 116 us (ML-KEM-512), 179 us (ML-KEM-768) and 271 us (ML-KEM-1024) in total on one core. The cost is dominated by SHAKE128 matrix expansion and by Keccak in general, which is why the larger security levels cost more mainly through the k^2 matrix. The most promising optimisation targets are a faster Keccak-f[1600] (unrolled or SIMD) and vectorised NTT arithmetic.

Implementation and report prepared with assistance from Claude (Anthropic, model Claude Sonnet 5.5).

Appendix A. Per-run medians (-O3)

Scheme	Operation	Run 1 (us)	Run 2 (us)	Run 3 (us)	Spread (max/min)
ML-KEM-512	KeyGen	48.47	33.12	32.16	1.51x
	Encaps	50.41	36.09	35.93	1.40x
	Decaps	59.26	46.99	46.91	1.26x
	Decaps (invalid ct)	46.42	46.83	46.43	1.01x
ML-KEM-768	KeyGen	51.82	60.01	51.52	1.16x
	Encaps	56.33	57.48	56.00	1.03x
	Decaps	72.27	70.80	70.52	1.02x
	Decaps (invalid ct)	70.83	97.31	72.25	1.37x
ML-KEM-1024	KeyGen	80.76	85.39	81.86	1.06x
	Encaps	83.84	86.15	84.81	1.03x
	Decaps	102.47	105.39	104.53	1.03x
	Decaps (invalid ct)	113.86	107.01	104.61	1.09x

Table A1. Median of 10,000 samples in each of three independent runs.

Appendix B. Reproducing the results

From the ML_KEM folder of the repository, on Linux, macOS or Windows with WSL (a C compiler and make are required):

```
make test          # 712 checks, incl. 60 known-answer vectors
make bench_mlkem
./bench_mlkem -n 10000 -w 500 --pin 1 --csv run1.csv
./bench_mlkem --level 768   # a single parameter set
make clean && make bench_mlkem OPT=-O2  # other optimisation levels
```

For stable results use the 'performance' CPU governor where available, pin to one core with --pin, close other workloads, and repeat the run several times as done here. The raw per-run output of the runs in this report is available as CSV files.

Implementation and report prepared with assistance from Claude (Anthropic, model Claude Sonnet 5.5).